```
# Install required libraries
!pip -q install ultralytics

from ultralytics import YOLO
from PIL import Image
import matplotlib.pyplot as plt

# Load YOLO classification model
classification_model = YOLO("yolov8n-cls.pt")

# Load sample image
!wget -q -O image.jpg https://ultralytics.com/images/bus.jpg

# Classification
results = classification_model("image.jpg")

# Display classification result
result = results[0]

print("Image Classification Result:")
print(result.probs)

# Display image
img = Image.open("image.jpg")
plt.figure(figsize=(8,6))
plt.imshow(img)
plt.axis("off")
plt.title("Image Classification")
plt.show()
```

```
━━━━━━━━━━━━━━━━━━━━ 46.6/46.6 kB 2.1 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 45.4 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━ 53.2/53.2 kB 3.9 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━ 77.7/77.7 kB 4.9 MB/s eta 0:00:00
tings v0.0.8 file ✅
```

```
!pip -q install ultralytics

from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2

# Load YOLOv8 pre-trained detection model
model = YOLO("yolov8n.pt")

# Download sample image
!wget -q -O bus.jpg https://ultralytics.com/images/bus.jpg

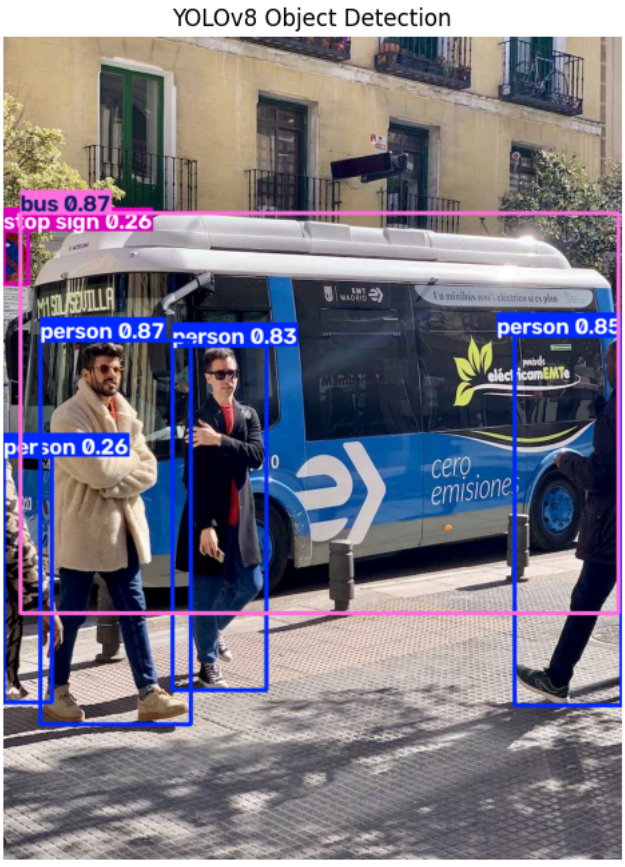
# Perform object detection
results = model("bus.jpg", conf=0.25)

# Get result
result = results[0]

# Display detected image
annotated_image = result.plot()

plt.figure(figsize=(12,8))
plt.imshow(cv2.cvtColor(annotated_image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("YOLOv8 Object Detection")
plt.show()
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.2.0/yolov8n.pt to 'yolov8n.pt': 100% 6.2MB/s 0.8s
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 214.5ms
Speed: 18.3ms preprocess, 214.5ms inference, 11.8ms postprocess per image at shape (1, 3, 640, 480)
```





```
for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
```

```
    class_name = model.names[class_id]

    print(
        "Object:",
        class_name,
        "| Confidence:",
        round(confidence, 2)
    )
```

```
Object: bus | Confidence: 0.87
Object: person | Confidence: 0.87
Object: person | Confidence: 0.85
Object: person | Confidence: 0.83
Object: person | Confidence: 0.26
Object: stop sign | Confidence: 0.26
```

```
import time
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Warm-up
model("bus.jpg", verbose=False)

# Measure inference time
start_time = time.time()

results = model("bus.jpg", verbose=False)

end_time = time.time()

inference_time = end_time - start_time

print("YOLOv8 Inference Time:", round(inference_time, 4), "seconds")
print("Approximate FPS:", round(1 / inference_time, 2))
```

```
YOLOv8 Inference Time: 0.183 seconds
Approximate FPS: 5.47
```

```
import pandas as pd

comparison = pd.DataFrame({
    "Model": ["YOLO", "SSD", "Faster R-CNN"],
    "Detection Type": ["Single-stage", "Single-stage", "Two-stage"],
    "Speed": ["Very Fast", "Fast", "Generally Slower"],
    "Typical Application": [
        "Real-time detection",
        "Real-time/embedded systems",
        "Accuracy-focused detection"
    ]
})

display(comparison)
```

	Model	Detection Type	Speed	Typical Application
0	YOLO	Single-stage	Very Fast	Real-time detection
1	SSD	Single-stage	Fast	Real-time/embedded systems
2	Faster R-CNN	Two-stage	Generally Slower	Accuracy-focused detection

Next steps: Generate code with `comparison` New interactive sheet

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

model = YOLO("yolov8n.pt")

results = model("bus.jpg")
result = results[0]

# Extract bounding boxes
for i, box in enumerate(result.boxes):
```

```
    coordinates = box.xyxy[0].tolist()
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])

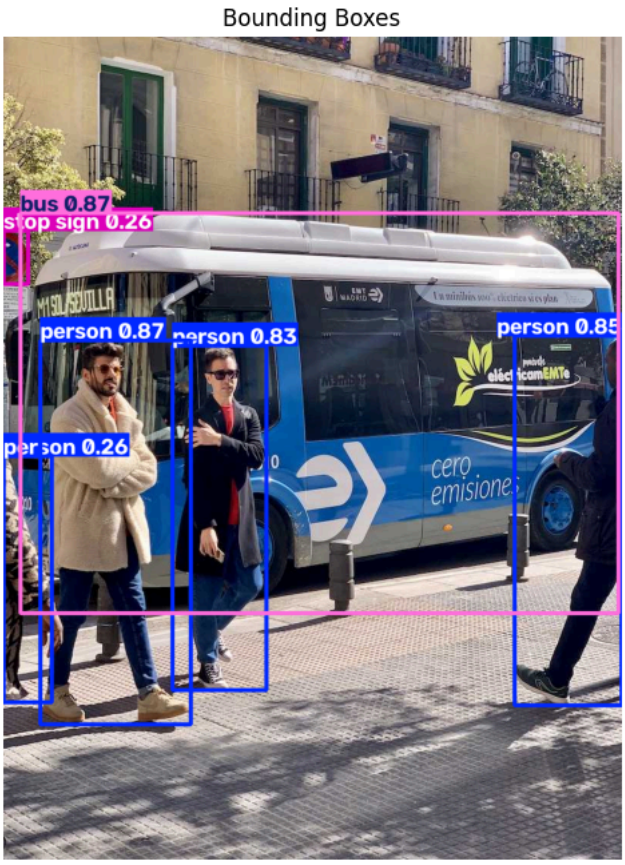
    x1, y1, x2, y2 = coordinates

    print(f"Object {i+1}")
    print("Class:", model.names[class_id])
    print("Confidence:", round(confidence, 2))
    print("Bounding Box:")
    print("x1 =", round(x1, 2))
    print("y1 =", round(y1, 2))
    print("x2 =", round(x2, 2))
    print("y2 =", round(y2, 2))
    print("-" * 40)
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 158.1ms
Speed: 6.0ms preprocess, 158.1ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480)
Object 1
Class: bus
Confidence: 0.87
Bounding Box:
x1 = 22.87
y1 = 231.28
x2 = 805.0
y2 = 756.84
----------------------------------------
Object 2
Class: person
Confidence: 0.87
Bounding Box:
x1 = 48.55
y1 = 398.55
x2 = 245.35
y2 = 902.7
----------------------------------------
Object 3
Class: person
Confidence: 0.85
Bounding Box:
x1 = 669.47
y1 = 392.19
x2 = 809.72
y2 = 877.04
----------------------------------------
Object 4
Class: person
Confidence: 0.83
Bounding Box:
x1 = 221.52
y1 = 405.8
x2 = 344.97
y2 = 857.54
----------------------------------------
Object 5
Class: person
Confidence: 0.26
Bounding Box:
x1 = 0.0
y1 = 550.53
x2 = 63.01
y2 = 873.44
----------------------------------------
Object 6
Class: stop sign
Confidence: 0.26
Bounding Box:
x1 = 0.06
y1 = 254.46
x2 = 32.56
y2 = 324.87
----------------------------------------
```

```
image = result.plot()

plt.figure(figsize=(12,8))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Bounding Boxes")
plt.show()
```




```
for box in result.boxes:

    class_id = int(box.cls[0])
    confidence = float(box.conf[0])

    print(
        "Object:",
        model.names[class_id],
        "| Confidence:",
        round(confidence, 4)
    )
```

```
Object: bus | Confidence: 0.8734
Object: person | Confidence: 0.8657
Object: person | Confidence: 0.8528
Object: person | Confidence: 0.8252
Object: person | Confidence: 0.2611
Object: stop sign | Confidence: 0.2551
```

```
def calculate_iou(box1, box2):

    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    intersection_width = max(0, x2 - x1)
    intersection_height = max(0, y2 - y1)

    intersection_area = (
        intersection_width * intersection_height
    )

    area1 = (
        (box1[2] - box1[0]) *
        (box1[3] - box1[1])
    )

    area2 = (
        (box2[2] - box2[0]) *
```

```
        (box2[3] - box2[1])
    )

    union_area = area1 + area2 - intersection_area

    if union_area == 0:
        return 0

    return intersection_area / union_area


# Example bounding boxes
box1 = [100, 100, 300, 300]
box2 = [150, 150, 350, 350]

iou = calculate_iou(box1, box2)

print("IoU =", round(iou, 4))
```

```
IoU = 0.3913
```

```
from ultralytics import YOLO

# Load pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

# Check model information
model.info()

print("Pre-trained YOLOv8 model loaded successfully.")
```

```
YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs
Pre-trained YOLOv8 model loaded successfully.
```

```
results = model("bus.jpg")

print("Detection completed using the pre-trained model.")

for box in results[0].boxes:

    class_id = int(box.cls[0])
    confidence = float(box.conf[0])

    print(
        model.names[class_id],
        "->",
        round(confidence, 2)
    )
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 160.5ms
Speed: 5.9ms preprocess, 160.5ms inference, 1.3ms postprocess per image at shape (1, 3, 640, 480)
Detection completed using the pre-trained model.
bus -> 0.87
person -> 0.87
person -> 0.85
person -> 0.83
person -> 0.26
stop sign -> 0.26
```

```
from ultralytics import YOLO

# Load YOLOv8 model trained on COCO
model = YOLO("yolov8n.pt")

# Display available COCO classes
print("Number of classes:", len(model.names))

print("\nCOCO Classes:")
for class_id, class_name in model.names.items():
    print(class_id, ":", class_name)
```

```
27 : tie
28 : suitcase
29 : frisbee
30 : skis
31 : snowboard
32 : sports ball
33 : kite
34 : baseball bat
35 : baseball glove
36 : skateboard
37 : surfboard
38 : tennis racket
39 : bottle
40 : wine glass
41 : cup
42 : fork
43 : knife
44 : spoon
45 : bowl
46 : banana
47 : apple
48 : sandwich
49 : orange
50 : broccoli
51 : carrot
52 : hot dog
53 : pizza
54 : donut
55 : cake
56 : chair
57 : couch
58 : potted plant
59 : bed
60 : dining table
61 : toilet
62 : tv
63 : laptop
64 : mouse
65 : remote
66 : keyboard
67 : cell phone
68 : microwave
69 : oven
70 : toaster
71 : sink
72 : refrigerator
73 : book
74 : clock
75 : vase
76 : scissors
77 : teddy bear
78 : hair drier
79 : toothbrush
```

```
results = model("bus.jpg")

result = results[0]

print("\nDetected Objects:")

for box in result.boxes:

    class_id = int(box.cls[0])
    confidence = float(box.conf[0])

    print(
        model.names[class_id],
        "Confidence:",
        round(confidence, 2)
    )
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 163.4ms
Speed: 4.5ms preprocess, 163.4ms inference, 1.3ms postprocess per image at shape (1, 3, 640, 480)

Detected Objects:
bus Confidence: 0.87
person Confidence: 0.87
person Confidence: 0.85
person Confidence: 0.83
person Confidence: 0.26
stop sign Confidence: 0.26
```

```
from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2

model = YOLO("yolov8n.pt")

results = model("bus.jpg")
result = results[0]

detected_objects = set()

for box in result.boxes:

    class_id = int(box.cls[0])
    class_name = model.names[class_id]

    detected_objects.add(class_name)

print("Objects detected:")
for obj in detected_objects:
    print("-", obj)
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 160.4ms
Speed: 7.6ms preprocess, 160.4ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480)
Objects detected:
- stop sign
- bus
- person
```

```
applications = [
    "Autonomous Vehicles",
    "Surveillance Systems",
    "Healthcare",
    "Retail Analytics",
    "Industrial Inspection"
]

print("Real-world applications of Object Detection:\n")

for i, application in enumerate(applications, 1):
    print(i, ".", application)
```

```
Real-world applications of Object Detection:

1 . Autonomous Vehicles
2 . Surveillance Systems
3 . Healthcare
4 . Retail Analytics
5 . Industrial Inspection
```

```
from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2

model = YOLO("yolov8n.pt")

thresholds = [0.10, 0.25, 0.50, 0.75]

for threshold in thresholds:

    results = model(
        "bus.jpg",
        conf=threshold,
        verbose=False
    )

    count = len(results[0].boxes)

    print(
        "Confidence Threshold:",
        threshold,
        "| Objects Detected:",
        count
    )
```

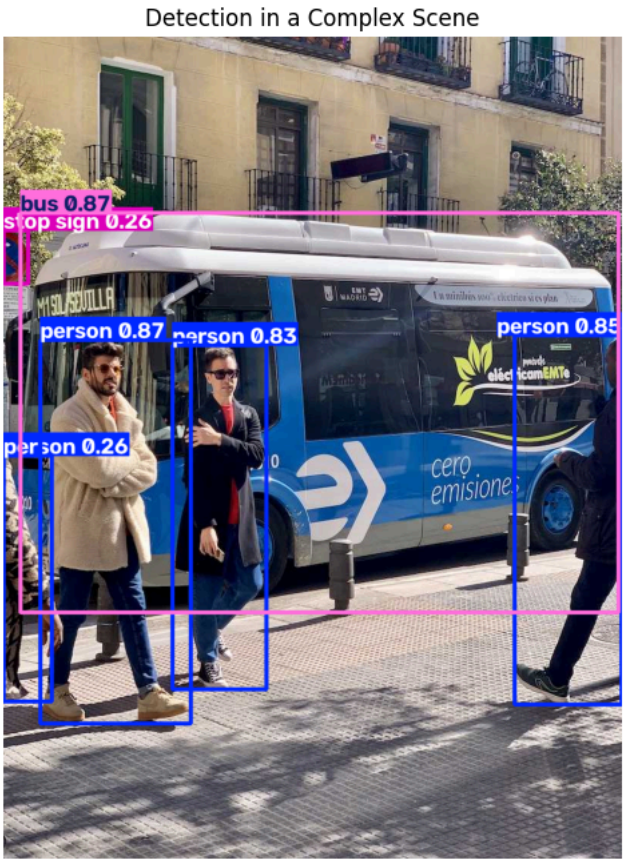
```
Confidence Threshold: 0.1 | Objects Detected: 6
Confidence Threshold: 0.25 | Objects Detected: 6
Confidence Threshold: 0.5 | Objects Detected: 4
Confidence Threshold: 0.75 | Objects Detected: 4
```

```
results = model("bus.jpg", conf=0.25)

annotated = results[0].plot()

plt.figure(figsize=(12,8))
plt.imshow(
    cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
)
plt.axis("off")
plt.title("Detection in a Complex Scene")
plt.show()
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 191.3ms
Speed: 4.6ms preprocess, 191.3ms inference, 1.7ms postprocess per image at shape (1, 3, 640, 480)
```




```
from ultralytics import YOLO
import time

models = [
    "yolov8n.pt",
    "yolov8s.pt"
]

for model_name in models:

    model = YOLO(model_name)

    start = time.time()

    results = model(
        "bus.jpg",
        verbose=False
    )

    end = time.time()
```

```
    inference_time = end - start

    print("\nModel:", model_name)
    print("Inference Time:",
          round(inference_time, 4),
          "seconds")

    print("Objects Detected:",
          len(results[0].boxes))
```

```
Model: yolov8n.pt
Inference Time: 0.2364 seconds
Objects Detected: 6
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt to 'yolov8s.pt': 100% ━━━━━━━━━━━━ 21.5MB

Model: yolov8s.pt
Inference Time: 0.5969 seconds
Objects Detected: 5
```

```
import pandas as pd

results_table = []

for model_name in ["yolov8n.pt", "yolov8s.pt"]:

    model = YOLO(model_name)

    start = time.time()
    results = model("bus.jpg", verbose=False)
    end = time.time()

    results_table.append({
        "Model": model_name,
        "Inference Time (sec)": round(end - start, 4),
        "Objects Detected": len(results[0].boxes)
    })

df = pd.DataFrame(results_table)

display(df)
```

	Model	Inference Time (sec)	Objects Detected
0	yolov8n.pt	0.2367	6
1	yolov8s.pt	0.6701	5

Next steps: Generate code with df | New interactive sheet

```
# ==========================================
# EXPERIMENT 9
# OBJECT DETECTION USING YOLOv8
# ==========================================

# 1. Install library
!pip -q install ultralytics

# 2. Import libraries
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import time

# 3. Load pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

# 4. Download sample image
!wget -q -O bus.jpg https://ultralytics.com/images/bus.jpg

# 5. Display input image
image = cv2.imread("bus.jpg")
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(10,7))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Input Image")
plt.show()

# 6. Perform object detection
start = time.time()

results = model(
    "bus.jpg",
    conf=0.25
)

end = time.time()

# 7. Get result
result = results[0]

# 8. Display detection result
annotated = result.plot()

plt.figure(figsize=(12,8))
plt.imshow(
    cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
)
plt.axis("off")
plt.title("YOLOv8 Object Detection Result")
plt.show()

# 9. Display detected objects
print("========== DETECTION RESULTS ==========")

for i, box in enumerate(result.boxes):

    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    coordinates = box.xyxy[0].tolist()

    print(
        f"{i+1}. Object: {model.names[class_id]}"
    )

    print(
        f"   Confidence: {confidence:.2f}"
    )

    print(
        f"   Bounding Box: "
        f"{[round(x, 2) for x in coordinates]}"
    )

# 10. Performance
inference_time = end - start

print("\n========== PERFORMANCE ==========")
print("Inference Time:",
      round(inference_time, 4),
      "seconds")

print("Approximate FPS:",
      round(1 / inference_time, 2))

print("\nTotal Objects Detected:",
      len(result.boxes))

print("\nExperiment completed successfully.")
```

Input Image




```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 166.2ms
Speed: 7.6ms preprocess, 166.2ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480)
```

YOLOv8 Object Detection Result

